進階程式設計
Advanced Program Design

L3

Chia-Chi Chang
ccchang@cc.ncue.edu.tw

同學提問是否有強制做評測系統的作業？

1. a.評測系統上有測試資料(比題目的範例複雜，當然測資是不提供的)，可以檢驗你的程式是否完備。

   b.期中考、期末考和2次小考會使用評測系統，所以選擇權在同學身上

2. 評測系統上的是「**練習題**」，無強制性，只給同學練習程式，不列入評分

3. 本學期的評分以2次小考、期中考和期末考為主，請看第一週投影片的描述(小考15%，期中考30%，期末考40%，其他15%)

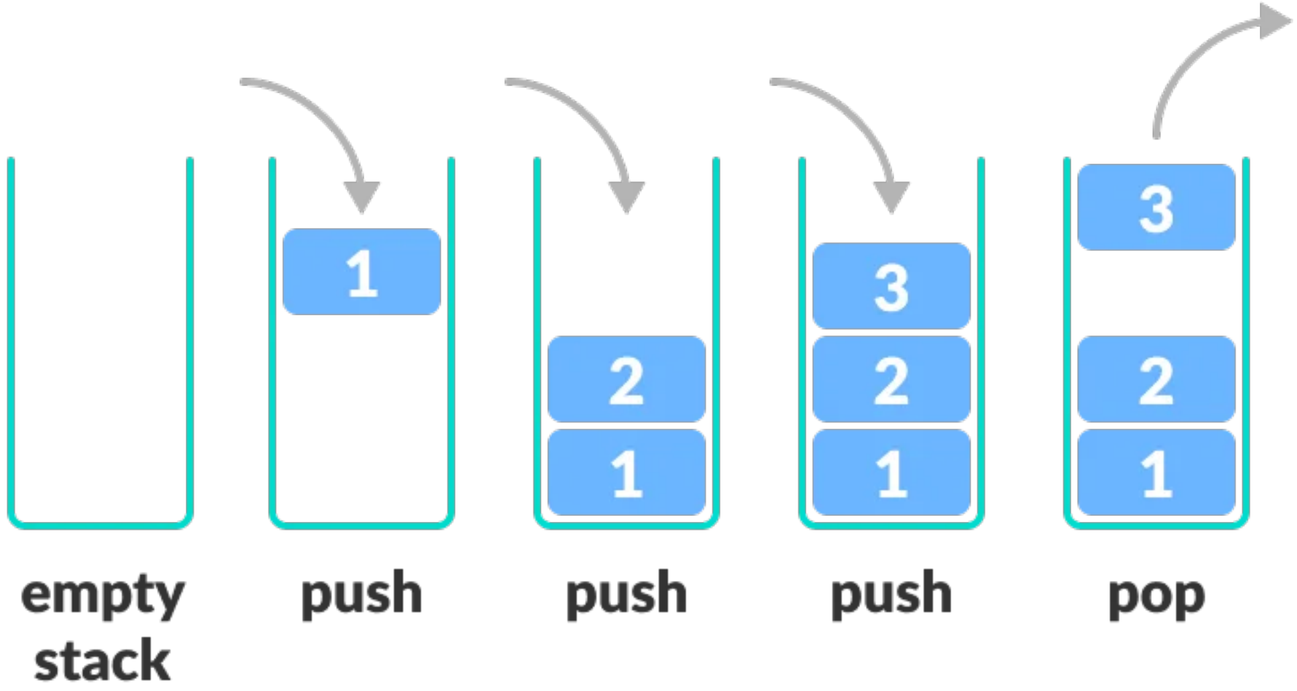
   ※過往同學繳交作業，幾乎都變成源自於幾個同學的版本或是網路版本，造成成績的公平性遭受質疑，因此本課程沒有所謂的作業，只讓同學去評測系統練習。

4. 所有的題目，都來自於老師從C語言教科書的作業練習題，沒有特定的書籍，至於考試題目就會以老師提供的練習題或投影片的內容為基礎去設計和延伸。

5. 若還有疑問，歡迎同學們來找老師(E-137)或助教(嵌入式系統暨無線感測網路實驗室E116)當面討論

堆疊(Stack)

- 堆疊(Stack)：資料後進先出(Last In First Out)
  - A stack is a linear data structure that follows the principle of Last In First Out (LIFO).
- 使用Push(推入、壓棧)和Pop(彈出、彈棧)
  - 資料推入堆疊區頂端(Add an element to the top of a stack)
  - 從堆疊區移除一個資料(Remove an element from the top of a stack)



函式呼叫

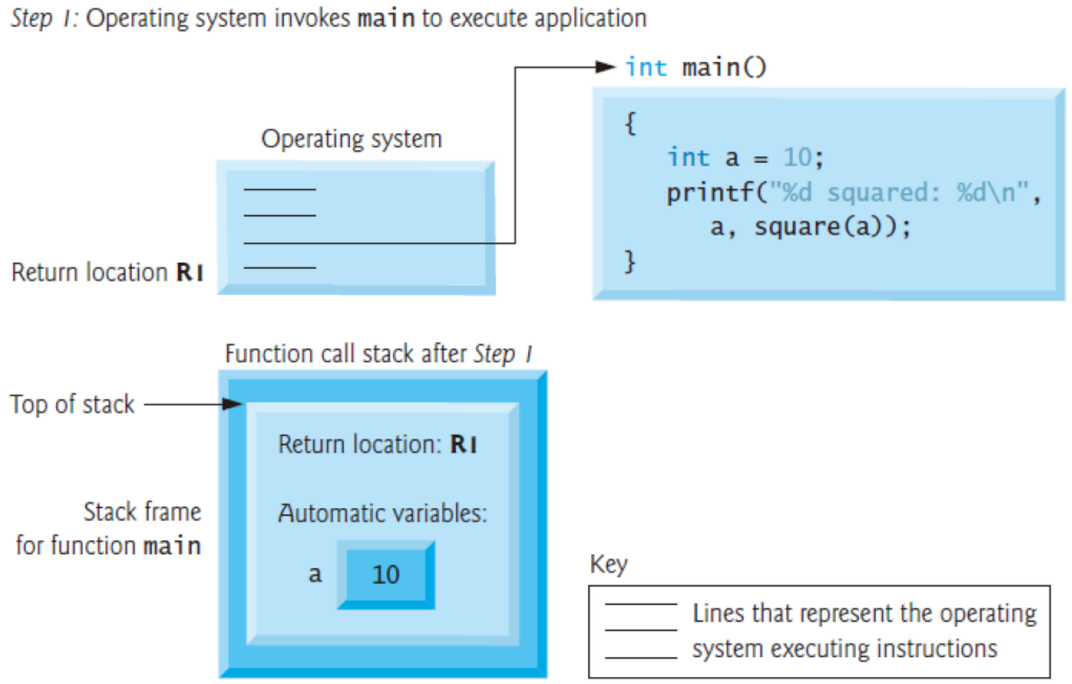


圖 5.7 作業系統調用 **main** 函式執行程式後的函式呼叫堆疊

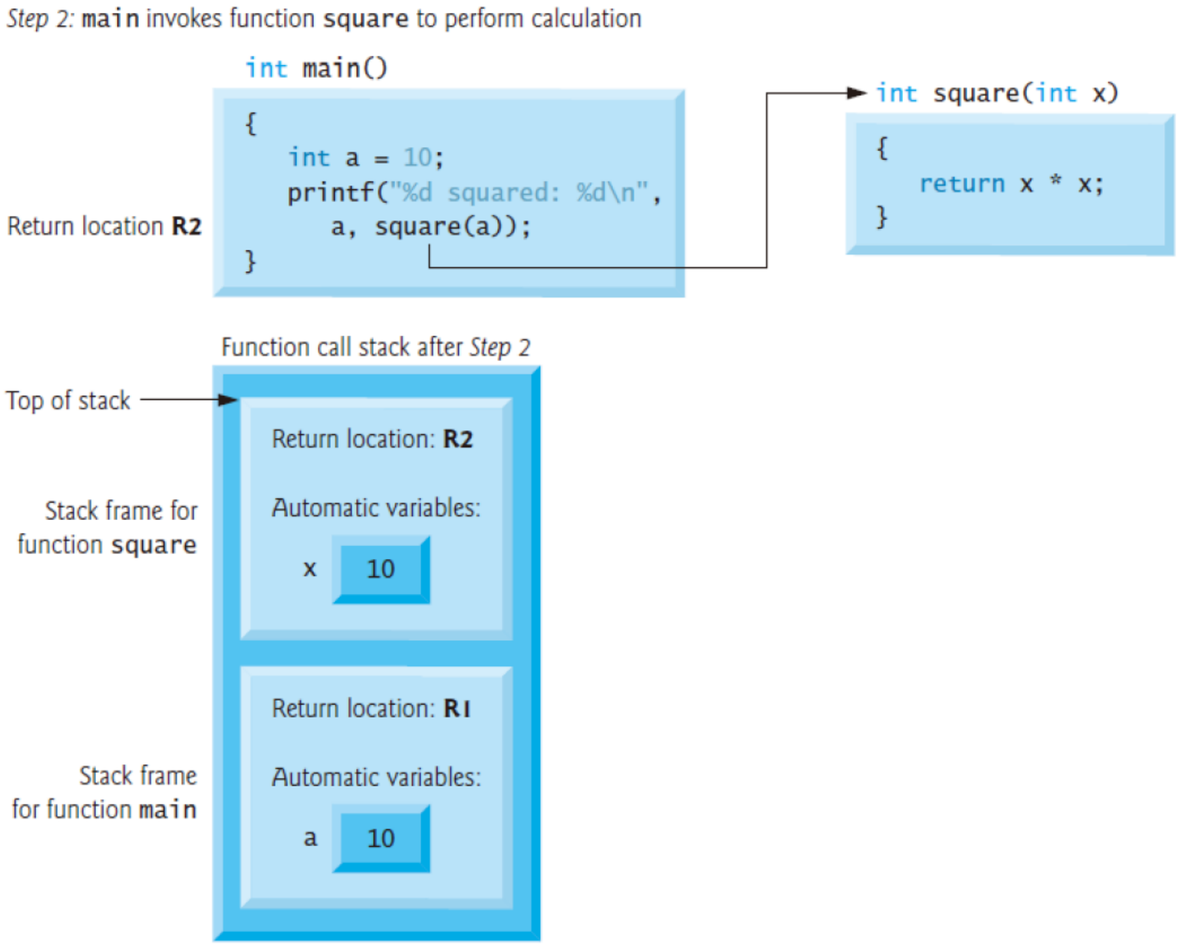


圖 5.8 **main** 函式調用 **square** 函式進行計算後的函式呼叫堆疊

函式呼叫

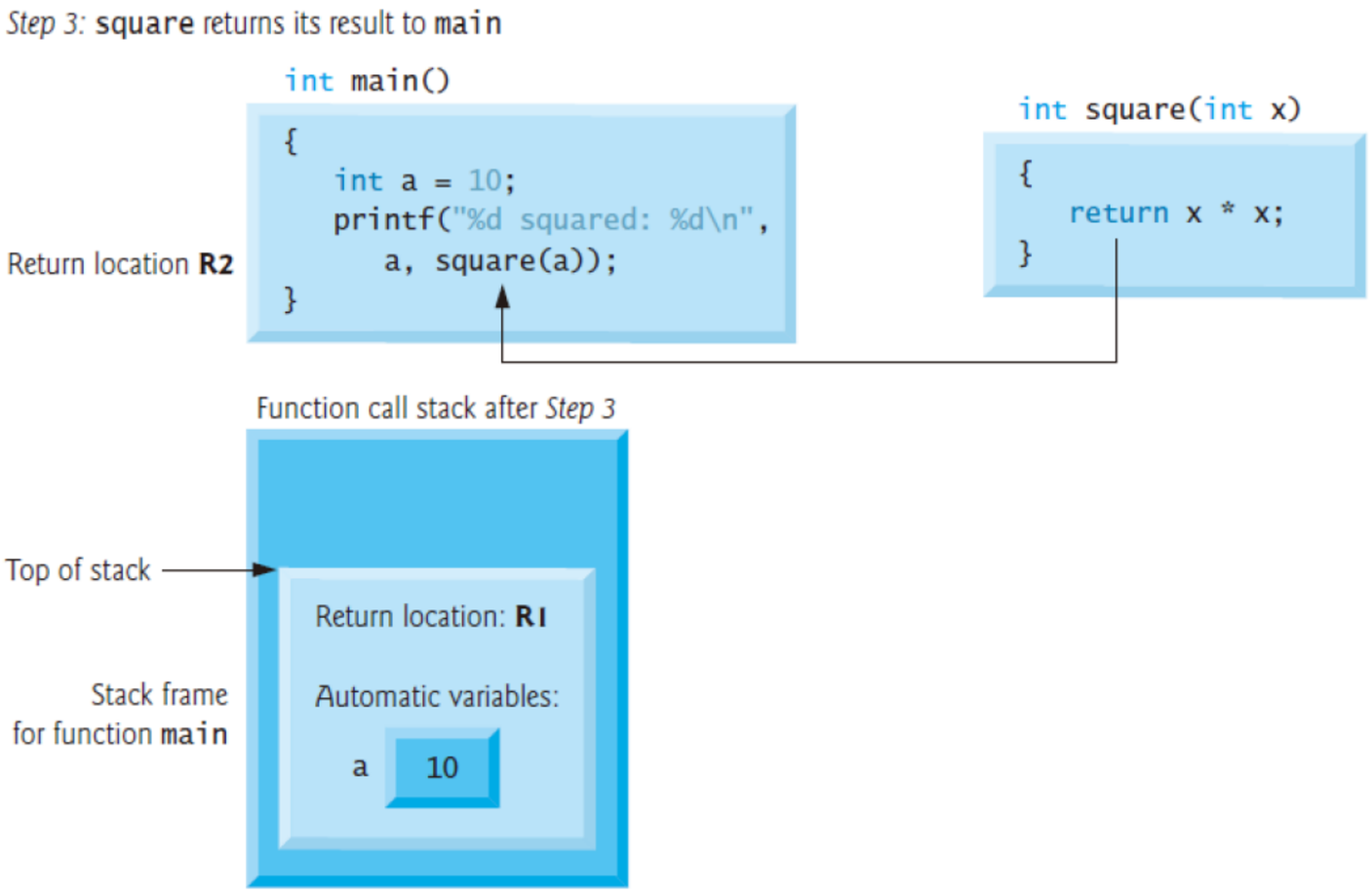


圖 5.9　**square** 函式回傳至 **main** 後的函式呼叫堆疊

計算1+2+3+…+10

1. 迴圈(for, while, do..while)
2. 遞迴(Recursion)：
   - 優點：有助於程式解題，把複雜題目簡單化
   - 缺點：設計不好就耗盡堆疊(stack)記憶體空間(因為不斷重複呼叫自己)，在微控制器裡的Embedded C compiler會限制或禁止。

```c
#include <stdio.h>
#include <stdlib.h>

int sum(int);
int main(int argc, char *argv[]) {
    int m;

    scanf("%d",&m);
    printf("sum = %d\n",sum(m));
    return 0;
}
int sum(int m) {
    int loop, sum = 0;

    for(loop=1; loop<=m; loop++) {
        sum += loop;
    }
    return sum;
}
```

迴圈

```c
#include <stdio.h>
#include <stdlib.h>

int sum(int);
int main(int argc, char *argv[]) {
    int m;

    scanf("%d",&m);
    printf("sum = %d\n",sum(m));
    return 0;
}
int sum(int m) {
    if(m>1) {
        return m + sum(m-1);
    } else {
        return 1;
    }
}
```

遞迴

指標(Pointer)

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int m = 20;
    int *ptr_m = &m;

    /*************************************
     *每1個變數都是放在記憶體，除了存放內容
     *之外，也應有「獨一無二」的位址
     *另外，每個變數都會有不同長度，佔據記憶體
     *空間也不一樣
     *ex. char佔據8-bit(1 Byte)
     *    short int佔據16-bit(2 Bytes)
     *    int佔據32-bit(4 Bytes)
     *************************************/

    //變數m的內容
    printf("The value of m is %d\n",m);
    //變數m在記憶體的位址(address)，我們習怪用16進制表示
    //0x是為了與10進制做區分，也有人在後面使用%XH或%Xh
    printf("The address of m is 0x%X\n",&m);
    //指標變數存放的內容是位址，在此是變數m的位址
    printf("The value of ptr_m is 0x%X\n",ptr_m);
    //指標變數也是放在記憶體中，因此指標變數也有自己的位址
    printf("The address of ptr_m is 0x%X\n",&ptr_m);
    return 0;
}
```

- 何謂指標？
  - 變數內容與記憶體位址(address)

```
The value of m is 20
The address of m is 0x62FE1C
The value of ptr_m is 0x62FE1C
The address of ptr_m is 0x62FE10

--------------------------------
Process exited after 0.03837 seconds with return value 0
Press any key to continue . . .
```

指標(Pointer)

➢ 如何使用指標變數？

```
01 #include <stdio.h>
02 #include <stdlib.h>
03
04 int main(int argc, char *argv[]) {
05     int m = 20;
06     int *ptr_m;                    //宣告1個指標變數ptr_m，未來會指向1個存放某個整數(int)
07     float *ptr_f;                  //宣告1個指標變數ptr_f，未來會指向1個存放某個浮點數(float)
08     int *backupKey;                //並不是只能1個指標變數指向1個變數(備用鑰匙概念)
09
10     ptr_m = &m;                    //指標變數ptr_m指向整數變數m，也可以解釋成指標變數的內容是變數m的位址
11                                      //此時，指標變數ptr_m與變數m有了連結關係
12     backupKey = ptr_m;             //將ptr_m的內容複製給backupKey(複製鑰匙)
13
14     printf("m = %d\n",m);          //使用直接存取(Direct addressing)
15     printf("m = %d\n",*ptr_m);     //使用間接存取(Indirect addressing)
16     printf("m = %d\n",*backupKey); //使用另一個指標變數backupKey去存取變數m
17     return 0;
18 }
```

指標(Pointer)

- 何時會用到指標？
  - 指標與函式的關係
    - 函式的傳值呼叫(Pass by Value)：void func(int m, int n);

```c
#include <stdio.h>
#include <stdlib.h>
void mySwap(int, int);
int main(int argc, char *argv[]) {
    int a = 20, b = 30;

    printf("a = %d, b = %d\n",a,b);
    mySwap(a,b);
    printf("a = %d, b = %d\n",a,b);
    return 0;
}
void mySwap(int m, int n) {
    int t;

    printf("Invoke the function of mySwap==\n");
    printf("m = %d, n = %d\n",m,n);
    printf("Swap two numbers...\n");
    t = m;
    m = n;
    n = t;
    printf("m = %d, n = %d\n",m,n);
    printf("==============================\n");
}
```

```
a = 20, b = 30
Invoke the function of mySwap==
m = 20, n = 30
Swap two numbers...
m = 30, n = 20
==============================
a = 20, b = 30
```

a和b的內容仍維持不變

```
--------------------------------
Process exited after 0.03015 seconds with return value 0
Press any key to continue . . .
```

指標(Pointer)

- 指標與函式的關係
  - 函式的傳參呼叫(Pass by Reference)：void func(int *ptrM, int *ptrN);
    - 傳址呼叫(Pass by Address)比較貼切(個人偏好)

```c
#include <stdio.h>
#include <stdlib.h>
void mySwap(int*, int*);
int main(int argc, char *argv[]) {
    int a = 20, b = 30;

    printf("a = %d, b = %d\n",a,b);
    mySwap(&a,&b);
    printf("a = %d, b = %d\n",a,b);
    return 0;
}
void mySwap(int* ptr_m, int* ptr_n) {
    int t;

    printf("Invoke the function of mySwap==\n");
    printf("*ptr_m = %d, *ptr_n = %d\n",*ptr_m,*ptr_n);
    printf("Swap two int pointers...\n");
    t = *ptr_m;
    *ptr_m = *ptr_n;
    *ptr_n = t;
    printf("*ptr_m = %d, *ptr_n = %d\n",*ptr_m,*ptr_n);
    printf("==============================\n");
}
```

```
a = 20, b = 30
Invoke the function of mySwap==
*ptr_m = 20, *ptr_n = 30
Swap two int pointers...
*ptr_m = 30, *ptr_n = 20
==============================
a = 30, b = 20
```

a和b的內容確實交換

```
--------------------------------
Process exited after 0.02766 seconds with return value 0
Press any key to continue . . .
```

判斷質數(Prime Number)

Q: 判斷輸入的整數是否為質數？

- 質數：除了1和自己之外，找不到任何數可以整除。

- 最簡易實做方法(不一定是最佳)：列舉法(Enumeration)、窮舉法、暴力法

1. https://www.youtube.com/watch?v=u40sQDBrGLs
2. 查表法(lookup table)：事先建立某個範圍的所有答案，有助於執行程式執行效率

10進制轉成2進制

```
01 #include <stdio.h>
02 #include <stdlib.h>
03
04 int main(int argc, char *argv[]) {
05     char a = 124; //0111 1100
06     int loop;
07
08     for(loop=0; loop<8; loop++) {
09         if(a & 0x80) {
10             printf("1_");
11         } else {
12             printf("0_");
13         }
14         a <<= 0x01;
15     }
16     return 0;
17 }
```

使用位元運算子&

左移1位等同於a *=2;

※左移比乘法來的有效率

The End